11420 AIA 5350 Deep Learning

Lab 5

111061220 林進成

---

1. Introduction

This report presents an integrated study of value-based reinforcement learning methods across Tasks 1–3. The goal is to understand how Deep Q-Networks (DQN) can be implemented and extended for environments of different complexity, starting from CartPole and then moving to Atari Pong. In Task 1, I implemented a vanilla DQN agent for CartPole using a fully connected neural network, experience replay, an epsilon-greedy policy, and a target network. In Task 2, I extended the same vanilla DQN framework to Pong by adding Atari frame preprocessing and a convolutional Q-network that receives stacked image frames as input. In Task 3, I further modified the Pong DQN agent with the three required enhancements: Double DQN, Prioritized Experience Replay (PER), and multi-step returns. These components were introduced to reduce Q-value overestimation, improve replay efficiency, and propagate reward information more quickly. The report is organized as follows: first, I describe the implementation of the baseline DQN framework and the three Task 3 enhancements; next, I present the experimental settings and evaluation protocol; then, I compare the results of Tasks 1–3 and analyze the contribution of each enhancement through ablation experiments; finally, I summarize the main findings and limitations.

2. Implementation

Vanilla DQN Framework

The basic algorithm used in Tasks 1 and 2 is Deep Q-Network (DQN). The purpose of DQN is to approximate the optimal action-value function $Q^*(s,a)$ with a neural network $Q_\theta(s,a)$. Given a transition $(s_t, a_t, r_{t+1}, s_{t+1}, d_t)$, the one-step DQN target is defined as

$$y_t = r_{t+1} + \gamma \max_{a'} Q_{\tilde{\theta}}(s_{t+1}, a')(1 - d_t),$$

where $Q_{\tilde{\theta}}$ is the target network and d_t indicates whether the episode has terminated. The Bellman error is then

$$\delta_t = y_t - Q_\theta(s_t, a_t).$$

The Q-network is trained by minimizing the squared Bellman error over sampled mini-batches from the replay buffer:

$$L(\theta) = \mathbb{E}[(y_t - Q_\theta(s_t, a_t))^2].$$

In the implementation, the current Q-value is selected with gather, while the target is computed without gradient tracking. The target network is periodically synchronized with the online network to stabilize bootstrapping. The core vanilla DQN update is implemented as follows:

```
q_values = self.q_net(states).gather(1, actions.unsqueeze(1)).squeeze(1)

with torch.no_grad():
    next_q_values = self.target_net(next_states).max(dim=1).values
    targets = rewards + self.gamma * next_q_values * (1.0 - dones)

loss = nn.MSELoss()(q_values, targets)

self.optimizer.zero_grad()
loss.backward()
torch.nn.utils.clip_grad_norm_(self.q_net.parameters(), max_norm=10.0)
self.optimizer.step()

if self.train_count % self.target_update_frequency == 0:
    self.target_net.load_state_dict(self.q_net.state_dict())
```

Task 1 uses a fully connected network because the CartPole state is a four-dimensional numerical vector. The network contains two hidden layers and outputs one Q-value for each action. Task 2 and Task 3 use image observations from Pong, so the input is first converted to grayscale, resized to 84×84, and stacked over four consecutive frames. The convolutional Q-network receives an input of shape $4 \times 84 \times 84$and outputs Q-values for all Pong actions.

```
self.network = nn.Sequential(
    nn.Conv2d(4, 32, kernel_size=8, stride=4),
    nn.ReLU(),
    nn.Conv2d(32, 64, kernel_size=4, stride=2),
    nn.ReLU(),
    nn.Conv2d(64, 64, kernel_size=3, stride=1),
    nn.ReLU(),
    nn.Flatten(),
    nn.Linear(64 * 7 * 7, 512),
    nn.ReLU(),
    nn.Linear(512, num_actions),
)

def forward(self, x):
    return self.network(x / 255.0)
```

The Atari preprocessor is responsible for converting raw RGB frames into the stacked state representation:

```
def preprocess(self, obs):
    gray = cv2.cvtColor(obs, cv2.COLOR_RGB2GRAY)
    resized = cv2.resize(gray, (84, 84), interpolation=cv2.INTER_AREA)
    return resized

def reset(self, obs):
    frame = self.preprocess(obs)
    self.frames = deque([frame for _ in range(self.frame_stack)], maxlen=self.frame_stack)
    return np.stack(self.frames, axis=0)

def step(self, obs):
    frame = self.preprocess(obs)
    self.frames.append(frame.copy())
    return np.stack(self.frames, axis=0)
```

The agent uses epsilon-greedy exploration. With probability ϵ, it samples a random action; otherwise, it selects the action with the highest predicted Q-value. The value of ϵ decays during training, so the agent gradually shifts from exploration to exploitation.

Double DQN

The first Task 3 enhancement is Double DQN. In vanilla DQN, the same target network is used to both select and evaluate the next action:

$$\max_{a'} Q_{\bar{\theta}}(s_{t+1}, a').$$

This maximization can produce overestimated Q-values because the maximum operator tends to select actions with positive estimation noise. Double DQN addresses this issue

```
def select_action(self, state):
    if random.random() < self.epsilon:
        return random.randint(0, self.num_actions - 1)

    state_tensor = torch.from_numpy(np.array(state)).float().unsqueeze(0).to(self.device)
    with torch.no_grad():
        q_values = self.q_net(state_tensor)

    return q_values.argmax().item()
```

by decoupling action selection and action evaluation. The online network selects the greedy next action:

$$a^* = \arg\max_{a'} Q_{\theta}(s_{t+1}, a'),$$

while the target network evaluates that selected action:

$$y_t^{\text{Double}} = r_{t+1} + \gamma Q_{\bar{\theta}}(s_{t+1}, a^*)(1 - d_t).$$

In the implementation, this is controlled by the `--use-double-dqn` flag. When it is enabled, the online network computes `next_actions`, and the target network computes the corresponding `next_q_values`:

```
with torch.no_grad():
    if self.use_double_dqn:
        next_actions = self.q_net(next_states).argmax(dim=1)
        next_q_values = self.target_net(next_states).gather(
            1, next_actions.unsqueeze(1)
        ).squeeze(1)
    else:
        next_q_values = self.target_net(next_states).max(dim=1).values

    targets = rewards + discounts * next_q_values * (1.0 - dones)
```

This modification keeps the training structure almost identical to DQN, but changes the bootstrap target so that action selection and action evaluation are no longer performed by the same network.

Prioritized Experience Replay

The second Task 3 enhancement is Prioritized Experience Replay (PER). Vanilla replay memory samples transitions uniformly, even though some transitions may provide more useful learning signals than others. PER assigns each transition a priority based on the magnitude of its Bellman error:

$$p_i = |\delta_i| + \varepsilon,$$

where ε is a small constant used to prevent zero priority. The sampling probability of transition i is

$$P(i) = \frac{p_i^{\alpha}}{\sum_k p_k^{\alpha}},$$

where α controls the strength of prioritization. Since prioritized sampling changes the training distribution, importance-sampling weights are used to reduce the resulting bias:

$$w_i = (NP(i))^{-\beta}.$$

The weights are normalized inside each mini-batch before being applied to the loss.

The replay buffer stores both transitions and priorities. When a new transition is inserted, it is assigned the current maximum priority so that new experiences have a chance to be sampled soon:

```
def add(self, transition, error=None):
    if error is None:
        if len(self.buffer) == 0:
            priority = 1.0
        else:
            priority = float(self.priorities[:len(self.buffer)].max())
            if priority <= 0:
                priority = 1.0
    else:
        priority = abs(float(error)) + 1e-6

    if len(self.buffer) < self.capacity:
        self.buffer.append(transition)
    else:
        self.buffer[self.pos] = transition

    self.priorities[self.pos] = priority
    self.pos = (self.pos + 1) % self.capacity
```

During sampling, priorities are converted into probabilities, and the sampled transitions are returned together with their indices and importance-sampling weights:

```
def sample(self, batch_size):
    current_size = len(self.buffer)
    priorities = self.priorities[:current_size]
    scaled_priorities = priorities ** self.alpha
    priority_sum = scaled_priorities.sum()

    if priority_sum <= 0:
        probabilities = np.ones(current_size, dtype=np.float32) / current_size
    else:
        probabilities = scaled_priorities / priority_sum

    indices = np.random.choice(current_size, batch_size, p=probabilities)
    samples = [self.buffer[idx] for idx in indices]

    weights = (current_size * probabilities[indices]) ** (-self.beta)
    weights /= weights.max()
    weights = weights.astype(np.float32)

    return samples, indices, weights
```

The element-wise squared Bellman error is multiplied by the importance-sampling weights:

```
elementwise_loss = 0.5 * td_errors.pow(2)
loss = (weights * elementwise_loss).mean()
```

After the update, the priorities of sampled transitions are refreshed using the latest absolute Bellman errors:

This allows transitions with larger prediction errors to be sampled more frequently

```
if self.use_per:
    self.memory.update_priorities(
        indices,
        td_errors.detach().abs().cpu().numpy()
    )
```

while still correcting part of the sampling bias through importance weighting.

Multi-step Return

The third Task 3 enhancement is the multi-step return. Instead of using only one reward before bootstrapping, an n-step return accumulates several future rewards before applying the target-network estimate. The standard one-step DQN target is

$$y_t^{(1)} = r_{t+1} + \gamma Q_{\bar{\theta}}(s_{t+1}, a').$$

With an n-step return, the target becomes

$$y_t^{(n)} = \sum_{k=0}^{n-1} \gamma^k r_{t+k+1} + \gamma^n Q_{\bar{\theta}}(s_{t+n}, a').$$

For this task, I used $n = 3$, so the reward term is

$$r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3}.$$

This helps propagate reward information faster than one-step DQN, which is useful in Pong because rewards are sparse and may occur only after a sequence of paddle movements.

The implementation maintains a temporary `n_step_buffer`:

```
self.n_step = args.n_step
self.n_step_buffer = deque(maxlen=args.n_step)
```

Each new transition is first appended to this buffer. Once enough transitions are available, or when the episode terminates, the code accumulates discounted rewards and constructs an aggregated transition:

The stored replay transition contains the accumulated reward and the effective discount

```
self.n_step_buffer.append((state, action, reward, next_state, done))

while len(self.n_step_buffer) >= self.n_step or (done and len(self.n_step_buffer) > 0):
    reward_sum = 0.0
    actual_n = min(self.n_step, len(self.n_step_buffer))
    next_state_n = self.n_step_buffer[actual_n - 1][3]
    done_n = self.n_step_buffer[actual_n - 1][4]

    for idx in range(actual_n):
        _, _, reward_i, candidate_next_state, candidate_done = self.n_step_buffer[idx]
        reward_sum += (self.gamma ** idx) * reward_i

        if candidate_done:
            actual_n = idx + 1
            next_state_n = candidate_next_state
            done_n = True
            break
```

γ^n, or a shorter discount if the episode ends before n steps:

```
state_0, action_0 = self.n_step_buffer[0][0], self.n_step_buffer[0][1]

self.add_to_memory(
    (
        state_0,
        action_0,
        reward_sum,
        next_state_n,
        done_n,
        self.gamma ** actual_n,
    )
)
```

Therefore, the training target can be written uniformly as:

$$y_t = R_t^{(n)} + \Gamma_t^{(n)} Q_{\bar{\theta}}(s_{t+n}, a')(1 - d_t),$$

where $R_t^{(n)}$ is the accumulated discounted reward and $\Gamma_t^{(n)}$ is the effective discount stored in the replay buffer.

Weight & Biases Tracking

I used Weight & Biases (W&B) to track model performance consistently across all three tasks. Since the three tasks differ in environment complexity and training length,

I used W&B not only to record the final reward, but also to monitor the training process, compare learning curves, and diagnose whether the Q-network was learning stably. For each task, I initialized a separate W&B project or run name so that the results could be compared without mixing different environments. Task 1 uses CartPole, while Tasks 2 and 3 use Pong, so I logged the task identity through the project name and the run name:

```
wandb.init(project="DLP-Lab5-Task1-CartPole", name=args.wandb_run_name, save_code=True)
```

```
wandb.init(project="DLP-Lab5-Task2-Pong", name=args.wandb_run_name, save_code=True)
```

```
wandb.init(
    project="DLP-Lab5-Task3-Pong",
    name=args.wandb_run_name,
    config=vars(args),
    save_code=True
)
```

In all tasks, I logged the episode reward as the basic indicator of training progress. This reward is useful for observing whether the agent is improving while interacting with the environment. I also logged the number of environment steps and update steps, so that the curves could be interpreted in terms of both sample collection and gradient updates:

```
wandb.log({
    "Episode": ep,
    "Total Reward": total_reward,
    "Env Step Count": self.env_count,
    "Update Count": self.train_count,
    "Epsilon": self.epsilon
})
```

For Task 1, the main performance indicator is the CartPole episode reward. Since CartPole episodes are short and the state space is simple, the training curve is mainly used to check whether the vanilla DQN agent learns to keep the pole balanced for longer periods. The logged Total Reward, Episode, Env Step Count, and Epsilon values show whether the agent improves as exploration decreases.

For Task 2 and Task 3, I used W&B more heavily because Pong has longer episodes and higher variance. In addition to the episode reward collected during training, I periodically evaluated the greedy policy over 20 fixed seeds and logged the average evaluation reward:

```
eval_reward = self.evaluate(episodes=20, seed_start=0)

wandb.log({
    "Env Step Count": self.env_count,
    "Update Count": self.train_count,
    "Eval Reward": eval_reward
})
```

The evaluation reward is more reliable than the raw training reward because it is computed using a fixed evaluation protocol and does not include epsilon-greedy exploration. Therefore, in the Pong tasks, I used Eval Reward as the main metric for comparing model snapshots and different experimental settings.

Also, I logged training diagnostics for the Q-network, including the loss, the mean and standard deviation of predicted Q-values, and the mean absolute Bellman error:

```
wandb.log({
    "Loss": loss.item(),
    "Q Mean": q_values.mean().item(),
    "Q Std": q_values.std().item(),
    "TD Abs Mean": td_errors.detach().abs().mean().item(),
    "Env Step Count": self.env_count,
    "Update Count": self.train_count
})
```

These metrics help explain whether a method is actually improving the value estimates or merely producing noisy reward fluctuations. For example, a decreasing loss and TD error suggest that the Q-network is fitting the Bellman targets more consistently, while abnormal Q-value statistics may indicate unstable value estimation. In Task 3, these diagnostics are especially useful for comparing the three enhancements because Double DQN, PER, and multi-step returns all affect the Bellman target or the replay distribution. Overall, W&B was used to track three levels of performance. First, Total Reward records how well the agent performs during training interaction. Second, Eval Reward

measures the greedy policy under a fixed testing protocol, which is the main metric for Pong. Third, optimization diagnostics such as Loss, Q Mean, Q Std, and TD Abs Mean provide additional evidence about training stability. By aligning these metrics with Env Step Count, I could compare Tasks 1–3 and analyze how each enhancement affected learning efficiency and final performance.

3. Analysis and Discussions

Task1

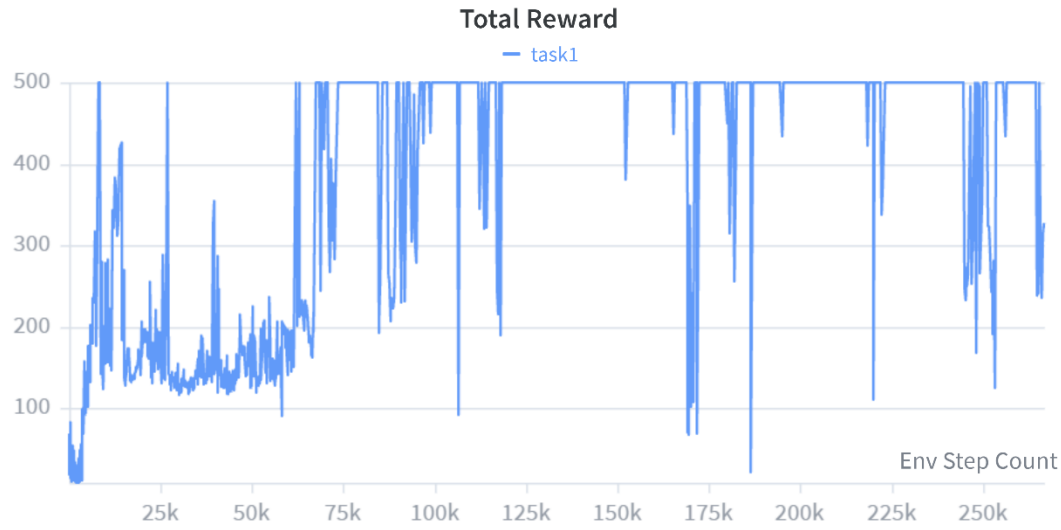

Figure 1. Training reward curve of Task 1 on CartPole.

In Task 1, the vanilla DQN agent was trained on CartPole-v1, where the objective is to keep the pole balanced for as long as possible. Since the environment gives a reward of +1at every time step and the maximum episode length is 500 steps, the highest possible episode reward is 500. Therefore, reaching a total reward close to 500 indicates that the agent has learned a near-optimal balancing policy. As shown in Figure 1, the training reward increases rapidly during the early stage, rising from very low rewards to several hundred steps within the first tens of thousands of environment steps. This indicates that the DQN agent quickly learned the basic relationship between cart movement and

pole stability. After roughly 70k–80k environment steps, the training reward frequently reaches 500, showing that the learned policy is capable of solving the task.

However, the training reward curve is not perfectly smooth. Even after the agent has already reached the maximum reward several times, the curve still contains occasional drops. This behavior is expected because the training policy remains epsilon-greedy, meaning that the agent still sometimes selects random actions for exploration. In CartPole, a few poor random actions can quickly destabilize the pole and cause a short episode. Therefore, the drops in Figure 1 do not necessarily mean that the learned Q-function has collapsed; rather, they reflect the stochasticity of the training policy and the sensitivity of CartPole to local action errors.

Figure 2. Evaluation reward curve of Task 1 on CartPole.

Figure 2 provides a clearer view of the actual policy quality because evaluation is performed using the greedy policy without exploration noise. The evaluation reward rises quickly and reaches the maximum score of 500 at around 75k environment steps. Although there is a temporary performance drop around 85k environment steps, the evaluation curve recovers and remains close to 500 for most later checkpoints. This shows that the DQN agent does not merely obtain high training reward by chance; it

learns a policy that generalizes across fixed evaluation seeds. The remaining occasional dips in evaluation are relatively limited and can be attributed to the instability of value-based learning, where small changes in Q-value estimates may temporarily change the greedy action selection.

Comparing Figure 1 and Figure 2, the evaluation curve is more informative for judging the final model performance. The training reward includes exploration and therefore remains noisy, while the evaluation reward measures the deterministic greedy policy. The fact that the evaluation reward repeatedly returns to 500 suggests that the learned Q-network has captured a robust control policy. This also confirms that the main DQN components used in Task 1—experience replay, target network updates, epsilon-greedy exploration, and Bellman-error minimization—are sufficient for solving CartPole.

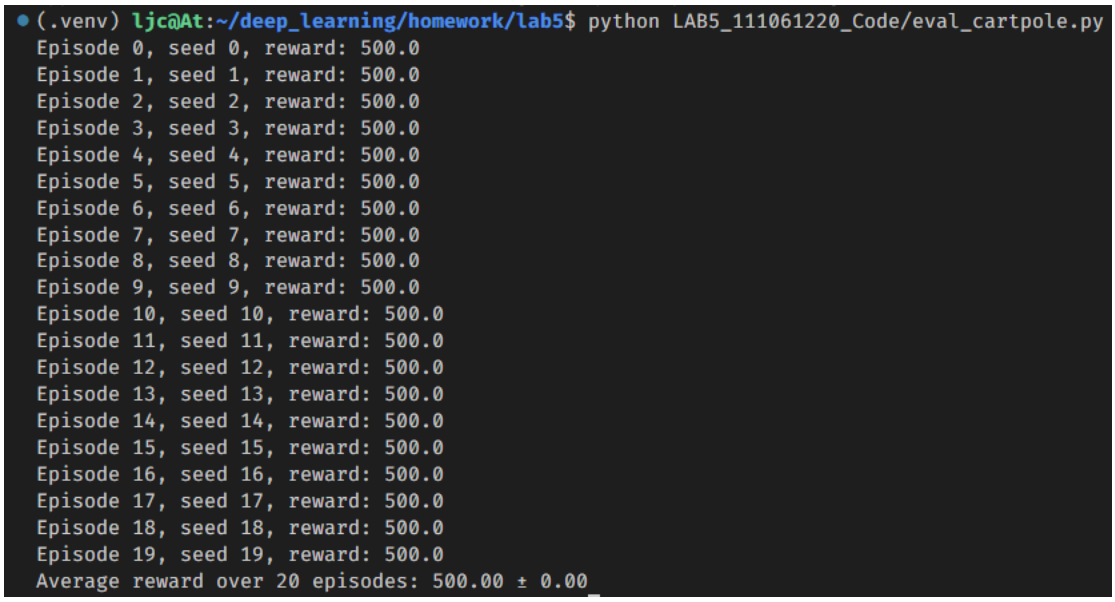

Figure 3. Final Task 1 evaluation over 20 fixed seeds.

The final evaluation result further confirms this conclusion. As shown in Figure 3, the trained model obtains a reward of 500.0 for all 20 evaluation episodes using seeds 0 through 19, resulting in an average reward of

$$500.00 \pm 0.00.$$

This means that the final policy consistently reaches the environment time limit across all tested seeds. Since CartPole-v1 terminates when the pole angle or cart position exceeds the allowed range, or when the episode reaches the 500-step time limit, receiving 500 reward in every evaluation episode indicates that the agent successfully avoids failure until the maximum episode length.

Overall, Task 1 demonstrates that the implemented vanilla DQN is effective on a low-dimensional control problem. The agent learns quickly, reaches the maximum possible evaluation reward, and remains stable across 20 fixed testing seeds. The remaining fluctuations in the training curve are mainly caused by epsilon-greedy exploration and do not affect the final greedy policy. Therefore, Task 1 serves as a reliable baseline showing that the replay buffer, target network, Q-learning update, and evaluation pipeline were implemented correctly before extending the method to the more difficult Pong tasks.

Task2

Figure 4. Training reward curve of Task 2 on Pong.

In Task 2, the vanilla DQN agent was trained on ALE/Pong-v5, which is significantly more difficult than CartPole because the input is no longer a compact numerical state

but a sequence of image frames. The agent observes raw Atari frames, preprocesses them into grayscale 84×84 images, and stacks four consecutive frames as the network input. The Q-network is therefore changed from the MLP used in Task 1 to a convolutional neural network with three convolutional layers and one fully connected layer before the final action-value output. This design allows the agent to extract spatial and temporal information from the Pong screen, such as the ball position, paddle position, and their recent movement. In the Pong environment, the agent controls the right paddle and receives positive reward when the ball passes the opponent's paddle, while losing reward when the ball passes its own paddle. The reduced Pong action space contains six discrete actions. The implementation follows this setup by using Atari preprocessing, frame stacking, and a CNN-based Q-network for image-based value prediction.

Figure 4 shows the training reward curve of Task 2. At the beginning of training, the total reward stays near -20, which means the agent almost always loses the game. This is expected because the initial Q-network is randomly initialized and the agent has not yet learned how paddle movements affect future rewards. During the first one million environment steps, the curve gradually increases but remains highly unstable, indicating that the agent is still exploring and only partially learning useful behaviors. Around 1.1M to 1.4M environment steps, the reward increases sharply from negative values to positive values. This phase transition suggests that the agent has learned a meaningful Pong strategy: it can track the ball and return it often enough to start winning points consistently. After this transition, the training reward mostly stays between approximately 15 and 20, although occasional drops still occur.

The fluctuations in Figure 4 are not necessarily a sign of failure. Unlike CartPole, Pong is a competitive Atari game with sparse rewards, long episodes, and high sensitivity to small action mistakes. In addition, the training policy is still epsilon-greedy, so random exploratory actions can interrupt an otherwise strong rally and cause a sudden loss of points. Therefore, the training reward is useful for observing the overall learning trend, but it is not the most reliable indicator of the final policy quality. The important observation is that the curve changes from a long negative-reward phase to a stable positive-reward phase, showing that the vanilla DQN agent successfully learns a functional control policy from image observations.

Figure 5. Evaluation reward curve of Task 2 on Pong.

Figure 5 shows the evaluation reward curve, which is more stable and more representative of the learned greedy policy. The evaluation reward begins near -21, then gradually improves as training progresses. Around one million environment steps, the evaluation reward crosses zero, and after approximately 1.3M to 1.5Menvironment steps, it rises to the range of about 15 to 18. From that point onward, the curve remains relatively stable and continues to fluctuate around a high positive reward. This indicates that the learned Q-network is not only improving during

exploratory training episodes, but also performs well when evaluated greedily without exploration noise.

The gap between the early negative reward and the later high positive reward highlights the difficulty of Pong compared with Task 1. In CartPole, the reward is dense because the agent receives a reward at every time step, so useful feedback is available immediately. In Pong, rewards are sparse and occur only when a point is scored, so the agent must learn from delayed consequences. The improvement in Figure 5 confirms that the replay buffer, target network, and convolutional Q-network are sufficient for vanilla DQN to learn a strong Pong policy, but the transition requires many more environment steps than CartPole.

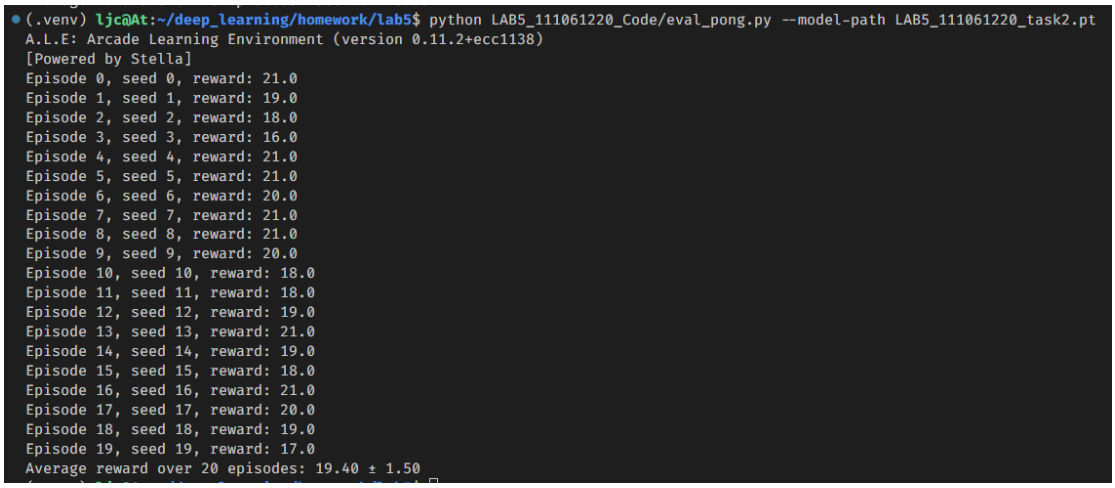

Figure 6. Final Task 2 evaluation over 20 fixed seeds.

The final evaluation result is shown in Figure 6. The trained Task 2 model was evaluated over 20 consecutive seeds, and the average reward was

$$19.40 \pm 1.50.$$

Most evaluation episodes achieve rewards between 18 and 21, with several episodes reaching 21. This shows that the final model can consistently beat the opponent across different initial conditions. The standard deviation is also relatively small, indicating

that the policy is not only strong in one lucky episode but is generally reliable across the 20-seed evaluation protocol.

Overall, Task 2 demonstrates that vanilla DQN can be extended from low-dimensional control to image-based Atari control by replacing the MLP with a CNN and applying frame preprocessing. Compared with Task 1, the learning process is slower and the training reward is noisier, but the final evaluation result confirms that the agent eventually learns a stable and high-performing Pong policy. The result also provides an important baseline for Task 3: before adding Double DQN, PER, and multi-step returns, the vanilla CNN-DQN is already capable of reaching strong performance, so the later ablation experiments should be interpreted relative to this Task 2 baseline.

Task3

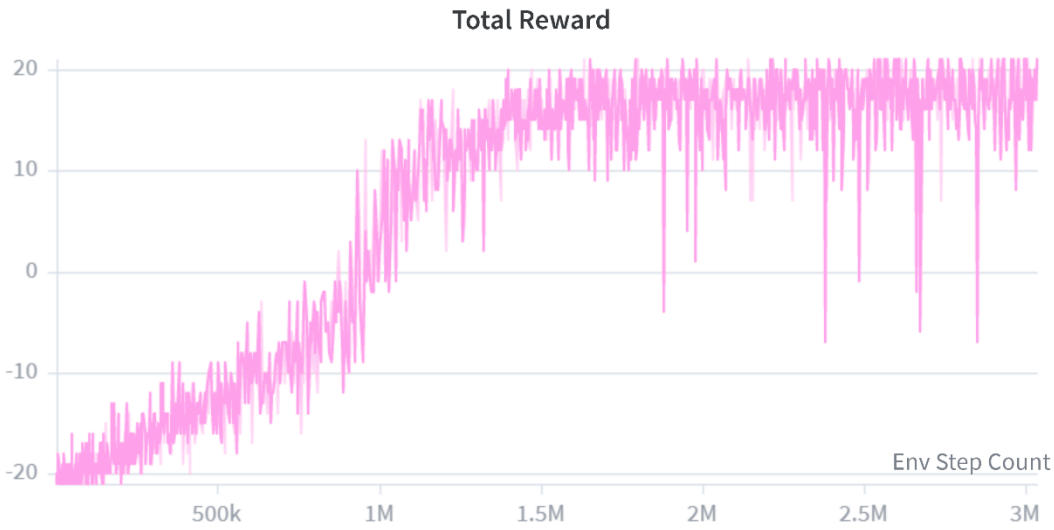

Figure 7. Training reward curve of Task 3 with all three enhancements enabled.

In Task 3, the Pong agent was trained with all three required enhancements enabled simultaneously: Double DQN, Prioritized Experience Replay (PER), and multi-step returns. Therefore, the result shown in this section should be interpreted as the performance of the combined enhanced DQN setting, rather than the best possible

configuration among all variants. Compared with Task 2, this setting modifies the target computation, the replay sampling distribution, and the reward target at the same time. Double DQN separates action selection from action evaluation to reduce overestimation, PER samples transitions with larger Bellman errors more frequently, and multi-step returns propagate reward information faster across several time steps. Since Pong gives $+1$reward when the ball passes the opponent and negative reward when the agent misses the ball, the episode reward roughly reflects how strongly the agent wins or loses a game. The full game score is bounded by the point-based Pong structure, where the maximum and minimum episode returns are approximately $+21$ and -21.

Figure 7 shows the training reward curve for the enhanced Task 3 model. At the beginning of training, the reward remains close to -20, meaning that the agent still loses almost every game. After several hundred thousand environment steps, the curve gradually improves, and a clear transition appears around 0.9M to 1.3M steps. During this period, the agent moves from a losing policy to a policy that can consistently obtain positive scores. After approximately 1.5M steps, the training reward mostly stays in the high positive range, often between 15 and 20. This indicates that the enhanced agent has learned a strong Pong policy and can win most training episodes. However, the training curve still contains occasional sharp drops, which is expected because Pong is sensitive to a small number of wrong actions and the training policy still includes exploration. Thus, the training reward should be used mainly to understand the learning trend rather than as the final evaluation metric.

Figure 8. Evaluation reward curve of Task 3 with all three enhancements enabled.

Figure 8 shows the evaluation reward curve, which is more important for judging the actual greedy policy. The evaluation reward starts near -21, then steadily increases as training proceeds. Around 1.0Msteps, the model begins to achieve positive evaluation rewards, and after around 1.4M to 1.6M steps, the curve reaches the range of approximately 15 to 18. The first point where the evaluation average exceeds 19 appears at 2.4M environment steps, where the logged evaluation reward reaches 19.5. This is important because the fixed saved checkpoints that I manually tested did not all exceed an average reward of 19, but the W&B evaluation curve shows that the model did reach the target level during training.

Figure 8 shows the evaluation reward curve, which is more important for judging the actual greedy policy. The evaluation reward starts near -21, then steadily increases as training proceeds. Around 1.0M steps, the model begins to achieve positive evaluation rewards, and after around 1.4M to 1.6M steps, the curve reaches the range of approximately 15 to 18. The first point where the evaluation average exceeds 19 appears around 2.4M environment steps, where the logged evaluation reward reaches

19.5. This is important because the fixed saved checkpoints that I manually tested did not all exceed an average reward of 19, but the W&B evaluation curve shows that the model did reach the target level during training.

The checkpoint-based evaluation results further illustrate this trend. At 600k steps, the model is still weak, with an average reward of

$$-5.50 \pm 2.73.$$

At 1M steps, the average improves to

$$8.35 \pm 2.92,$$

showing that the agent has begun to learn useful ball-tracking behavior. At 1.5M and 2M steps, the performance reaches

$$16.80 \pm 1.89$$

and

$$16.60 \pm 2.99,$$

respectively. These results show that the policy is already strong, but not yet consistently above the target threshold. At 2.5M steps, the checkpoint achieves

$$18.50 \pm 1.57,$$

which is close to the required average score of 19 but still slightly below it. The best saved model, however, reaches

$$19.70 \pm 1.19,$$

showing that the combined enhanced DQN is capable of reaching a high-performing policy when the best evaluation checkpoint is selected.

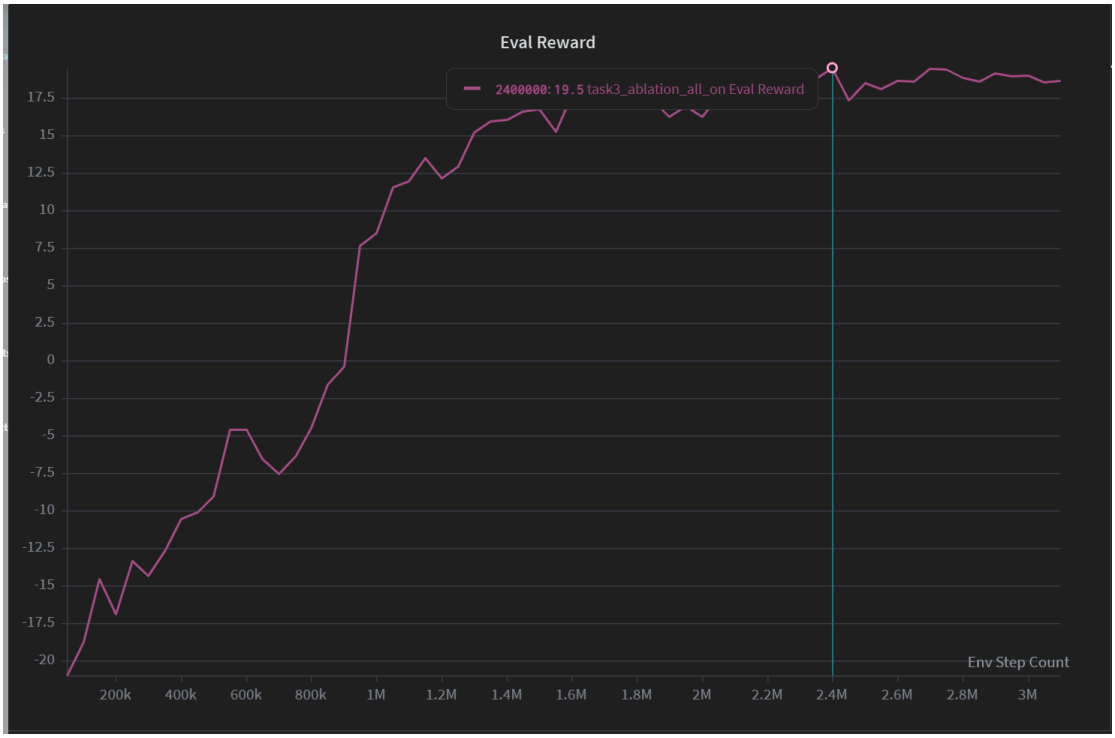

Figure 9. The first environmental step where the model got average reward >= 19

```
(.venv) ljc@At:~/deep_learning/homework/lab5$ python LAB5_111061220_Code/eval_pong.py --model-path LAB5_111061220_task3_600000.pt
A.L.E: Arcade Learning Environment (version 0.11.2+ecc1138)
[Powered by Stella]
Episode 0, seed 0, reward: -7.0
Episode 1, seed 1, reward: -1.0
Episode 2, seed 2, reward: -5.0
Episode 3, seed 3, reward: -3.0
Episode 4, seed 4, reward: -5.0
Episode 5, seed 5, reward: 1.0
Episode 6, seed 6, reward: -6.0
Episode 7, seed 7, reward: -6.0
Episode 8, seed 8, reward: -4.0
Episode 9, seed 9, reward: -2.0
Episode 10, seed 10, reward: -10.0
Episode 11, seed 11, reward: -6.0
Episode 12, seed 12, reward: -8.0
Episode 13, seed 13, reward: -8.0
Episode 14, seed 14, reward: -8.0
Episode 15, seed 15, reward: -7.0
Episode 16, seed 16, reward: -6.0
Episode 17, seed 17, reward: -3.0
Episode 18, seed 18, reward: -7.0
Episode 19, seed 19, reward: -9.0
Average reward over 20 episodes: -5.50 ± 2.73
```

Figure 10. Task 3 evaluation over 20 fixed seeds at 600k steps.

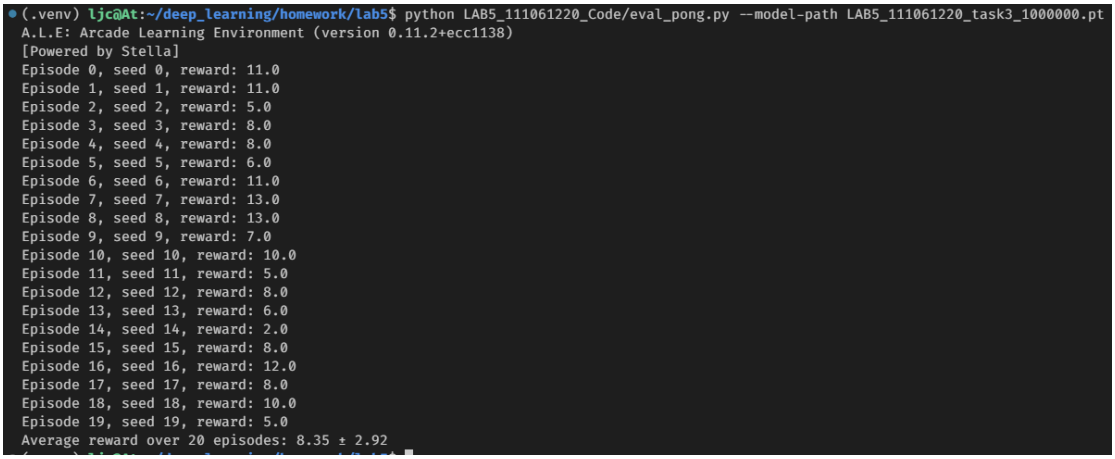

Figure 11. Task 3 evaluation over 20 fixed seeds at 1M steps.

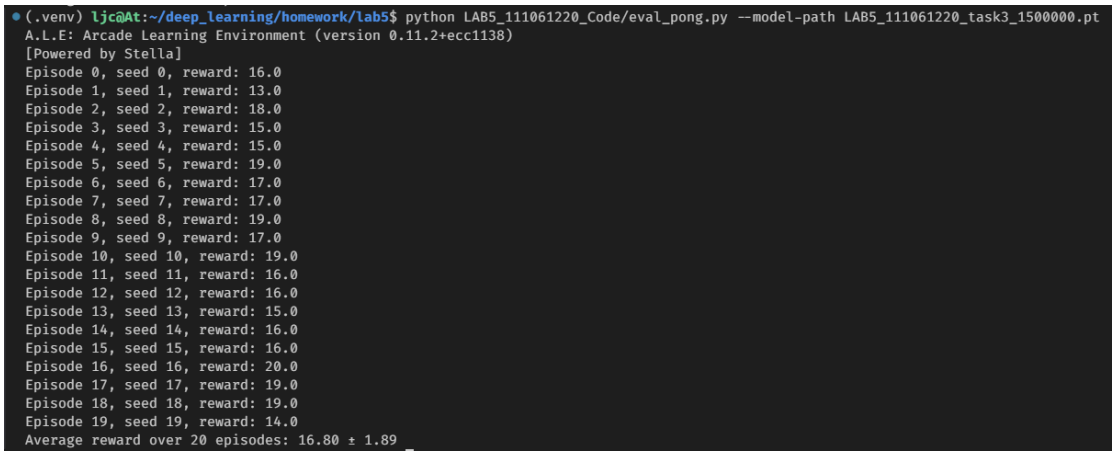

Figure 12. Task 3 evaluation over 20 fixed seeds at 1.5M steps.

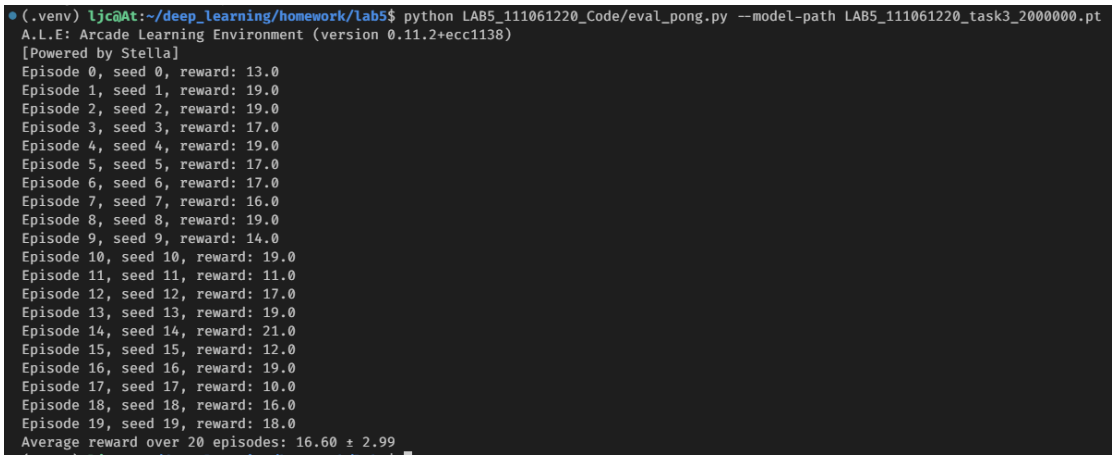

Figure 13. Task 3 evaluation over 20 fixed seeds at 2M steps.

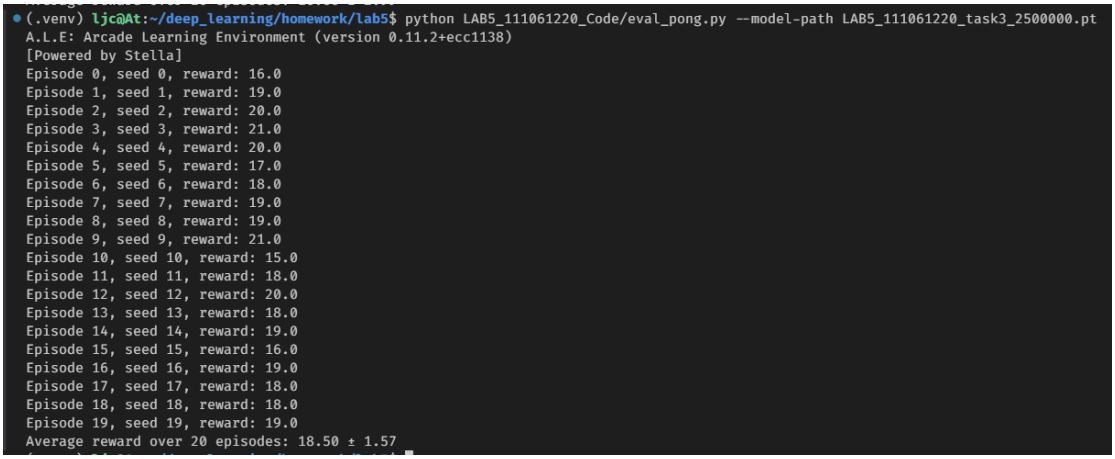

Figure 14. Task 3 evaluation over 20 fixed seeds at 2.5M steps.

It is also important to clarify the evaluation protocol. I did not search for a favorable seed range to artificially increase the score. For Task 3, as in Tasks 1 and 2, I used the same reference evaluation seeds from 0 to 19. Therefore, the reported results reflect the model's performance under a fixed and consistent 20-seed testing protocol. The final best-model result of 19.70 ± 1.19 indicates that most evaluation episodes achieve scores close to 20 or 21, meaning that the learned policy can reliably beat the built-in opponent across different initial conditions.

Overall, Task 3 shows that combining Double DQN, PER, and multi-step returns can train a strong Pong agent, but the learning curve is not immediately superior at every checkpoint. The model requires a long training horizon before the combined enhancements produce a stable high-performing policy. In particular, the 600k and 1M checkpoints are still far from the final result, while the model becomes competitive after approximately 1.5M steps and first exceeds the average reward threshold of 19 around 2.4M steps. This suggests that the three enhancements improve the final learning capability of the agent, but they do not guarantee faster early convergence under every checkpoint. The result should therefore be interpreted as the performance of the all-

enhancement Task 3 model, with the individual contribution of each enhancement analyzed separately in the following ablation section.

Ablation Study

Figure 15. Best evaluation reward curves of Task 3 ablation experiments.

To analyze the contribution of each Task 3 enhancement, I conducted an ablation study with five settings: all enhancements disabled, Double DQN only, PER only, multi-step return only, and all three enhancements enabled. In this experiment, the three enhancements refer specifically to Double DQN, Prioritized Experience Replay (PER), and multi-step return. The curves in Figure 15 show the best evaluation reward achieved so far during training. This representation is useful because it highlights how quickly each method reaches a strong policy, even if later checkpoints fluctuate.

The overall result shows that all methods eventually learn a strong Pong policy, but their learning speed and stability differ. Among the single-enhancement settings, **Double DQN only** and **multi-step return only** are the most effective. Double DQN only reaches the target threshold of average reward 19 at around 1.7M environment steps, while multi-step return only reaches the same threshold at around 1.85M steps.

In contrast, **PER only** improves much more slowly and does not reach an average reward of 19 within the visible 3M-step range. The all-off baseline also eventually reaches the threshold, but only at around 2.75M steps. The all-on model reaches 19.5 at around 2.4M steps, confirming that the combined enhanced model is capable of reaching the required score, but it is not the fastest among all ablation settings.

Setting	**First Eval ≥ 15**	**First Eval ≥ 18**	**First Eval ≥ 19**	**Best Eval ≤ 3M**
All off	1.40M	1.50M	2.75M	19.15
Double DQN only	**1.25M**	**1.70M**	**1.70M**	19.55
PER only	1.55M	2.60M	Not reached	18.65
Multi-step only	1.35M	1.70M	1.85M	**19.90**
All on	1.30M	1.70M	2.40M	19.50

Table 1. Ablation summary based on evaluation reward thresholds.

From Table 1, the most important observation is that the three enhancements do not contribute equally. Multi-step return gives the best final result within 3M steps, reaching a best evaluation reward of 19.90. Double DQN only gives the fastest threshold crossing, reaching 19 at 1.7M steps. PER only is the weakest individual enhancement in this experiment. Although PER is designed to replay more informative transitions more frequently and improve learning efficiency, its isolated effect here is limited; the PER-only run improves later than the others and remains below the 19-point threshold within the plotted range.

The all-on setting performs well, but it does not dominate the single-enhancement settings. This suggests that combining enhancements can introduce interactions that are not purely additive. For example, PER changes the replay distribution, while multi-step return changes the target value stored in the replay buffer. When both are enabled, the

TD error used for prioritization is affected by a longer-horizon target, which may make the replay distribution more aggressive or noisier during some training phases. Therefore, although the all-on model eventually reaches strong performance, the ablation result shows that the fastest improvement comes from simpler combinations, especially Double DQN only and multi-step return only.

Figure 16. Evaluation reward curves of Task 3 ablation experiments.

Figure 16 shows the non-cumulative evaluation reward at each evaluation point. Compared with Figure 15, this curve better reflects stability. The all-on model improves from negative reward to positive reward around 0.9M–1.0M steps and then reaches the 15–18 range around 1.3M–1.7M steps. However, its instantaneous evaluation reward still fluctuates after reaching high performance. This confirms that the all-on model learns a strong policy, but the learned policy is not monotonically improving at every checkpoint.

The Double DQN only curve is relatively strong in the middle stage. It reaches high evaluation scores earlier than most other settings and crosses 19 around 1.7M steps. This supports the idea that reducing overestimation bias is useful in Pong, where small

Q-value ranking errors can change the selected paddle action. Multi-step return only also performs strongly and eventually reaches the highest best score. This result is reasonable because Pong rewards are delayed: the effect of a paddle movement may only be reflected several frames later when the point is won or lost. Multi-step return helps propagate this delayed reward signal faster than the one-step target.

PER only shows a different pattern. It improves more slowly and reaches a lower final level than the other enhanced settings. This does not mean PER is incorrect, but it suggests that prioritizing transitions by TD error alone is not sufficient to provide the largest improvement in this Pong setup. Since Pong has sparse rewards and many visually similar states, high TD-error transitions may not always correspond to the most useful policy-improving transitions. As a result, PER alone improves replay efficiency less clearly than expected.

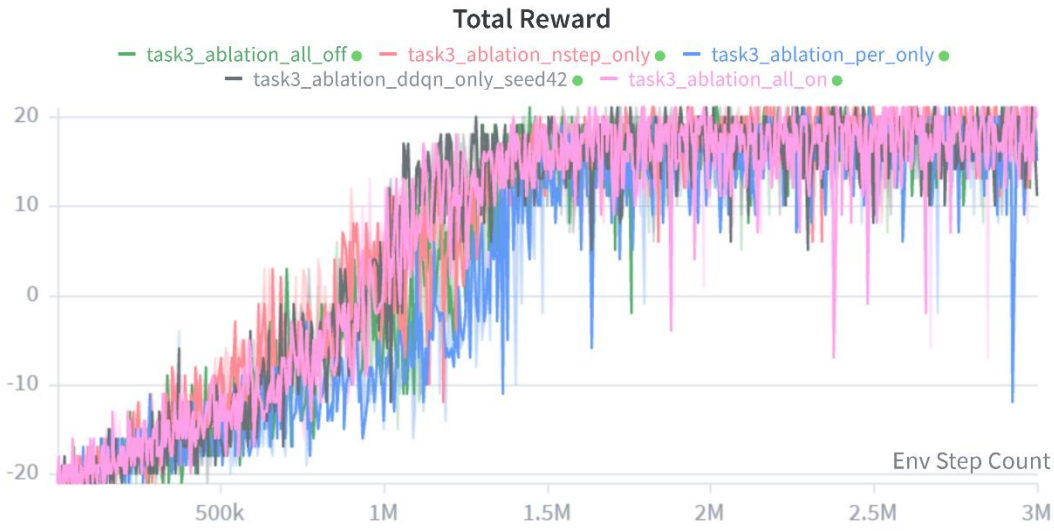

Figure 17. Training reward curves of Task 3 ablation experiments.

Figure 17 compares the training reward curves. All settings start near -20, meaning that all agents initially lose almost every game. The curves then gradually improve and enter a transition phase around 0.8M–1.4M environment steps. After this transition,

most settings produce positive training rewards and eventually stabilize near the high positive range. The total reward curves are noisier than the evaluation curves because training still involves exploration. Nevertheless, the training curves show that all methods eventually learn meaningful Pong behavior.

To summarize the training reward behavior, I computed the mean training reward over different environment-step windows.

Setting	0–500k	500k–1M	1M–1.5M	1.5M–2M	2M–3M
All off	-18.38	-8.75	7.77	16.15	17.30
Double DQN only	-18.62	-7.63	**12.67**	**16.95**	17.40
PER only	-18.39	-12.34	0.53	14.08	16.15
Multi-step only	-17.75	**-4.50**	9.66	16.82	**18.36**
All on	-17.81	-7.73	12.22	16.61	17.09

Table 2. Mean training reward over different environment-step windows.

Table 2 shows that multi-step return has the best training reward in the 500k–1M window and the 2M–3M window, while Double DQN has the strongest average reward in the 1M–2M range. PER only is consistently weaker, especially between 500k and 1.5M. This supports the same conclusion as the evaluation curves: multi-step return improves reward propagation and final performance, Double DQN improves mid-stage learning efficiency, and PER alone contributes the least in this setting.

Sample Efficiency Analysis

The sample efficiency of a value-based RL method can be evaluated by asking how many environment interactions are needed before the model reaches a certain evaluation reward. In this report, I use environment step count as the main x-axis because all Task 3 runs are trained and evaluated according to environment steps. This

is more appropriate than episode count because Pong episode length changes significantly as the policy improves.

The first observation is that all methods have a long initial learning phase. Before 500k steps, none of the methods has learned a strong policy; the evaluation reward remains negative for all settings. This indicates that Atari Pong requires substantially more samples than CartPole because the agent must learn from high-dimensional visual input and sparse point-based rewards. The main differences between methods appear after 500k steps.

The first major threshold is reaching non-negative evaluation reward. Multi-step return only reaches this point earliest at around 650k steps, followed by Double DQN only at around 750k steps. The all-on model reaches non-negative performance at around 950k steps, while all-off requires about 1.05M steps and PER only requires about 1.25M steps. This shows that multi-step return provides the clearest early sample-efficiency gain. It allows reward information to propagate over multiple steps, so the agent can connect paddle actions with future point outcomes more quickly.

Setting	**First Eval ≥ 0**	**First Eval ≥ 10**	**First Eval ≥ 15**	**First Eval ≥ 19**
All off	1.05M	1.25M	1.40M	2.75M
Double DQN only	0.75M	**1.05M**	**1.25M**	**1.70M**
PER only	1.25M	1.45M	1.55M	Not reached
Multi-step only	**0.65M**	1.20M	1.35M	1.85M
All on	0.95M	**1.05M**	1.30M	2.40M

Table 3. Sample efficiency comparison by threshold-crossing step.

Table 3 shows that Double DQN only is the most sample-efficient method for reaching the final threshold of 19. It reaches the threshold at 1.70M steps, which is about 1.05M steps earlier than the all-off baseline. Multi-step return only is also strong, reaching 19 at 1.85M steps. The all-on setting reaches the target later, at 2.40M steps. Therefore, even though the all-on model is the official combined-enhancement setting, it is not the most sample-efficient configuration in this ablation.

This result suggests that adding more enhancements does not automatically improve sample efficiency. In particular, combining PER with the other two enhancements may introduce more complicated learning dynamics. PER changes which transitions are replayed; multi-step return changes the target associated with those transitions; Double DQN changes the bootstrap value. These modifications are individually reasonable, but their combination may require more training before stabilizing. This explains why the all-on setting eventually reaches a high evaluation reward but crosses the 19-point threshold later than Double DQN only and multi-step only.

Another useful view is to compare evaluation reward at fixed checkpoints.

Setting	**600k**	**1M**	**1.5M**	**2M**	**3M**
All off	-8.30	-0.55	18.05	17.70	18.45
Double DQN only	-8.50	**8.90**	16.25	**19.25**	17.95
PER only	-11.80	-7.75	14.60	16.05	17.80
Multi-step only	**-3.20**	5.70	14.50	18.05	**19.40**
All on	-4.60	8.50	16.75	16.25	19.00

Table 4. Evaluation reward at fixed environment-step checkpoints.

At 600k steps, all methods are still below the target, but multi-step return only and all-on are already less negative than the others. At 1M steps, Double DQN only and all-on are clearly ahead, both reaching around 8–9 average reward. At 1.5M steps, all-off, Double DQN only, and all-on are already in the 16–18 range, while PER only and multi-step only lag slightly behind. At 2M and 3M steps, most methods converge to a high reward range, but the relative order still varies. This reinforces the idea that the best method depends on the training horizon: Double DQN is strong in the middle stage, multi-step return is strong in the long run, and all-on eventually reaches the target but does not dominate throughout training.

Overall, the sample-efficiency analysis leads to three conclusions. First, **multi-step return provides the strongest early reward-propagation benefit**, especially before 1M steps. Second, **Double DQN gives the fastest path to the 19-point evaluation threshold**, suggesting that reducing Q-value overestimation is especially helpful once the agent begins to learn a meaningful Pong policy. Third, **PER alone is not sample-efficient in this experiment**, since it improves slowly and does not reach the 19-point threshold within the plotted range. The all-on model confirms that the three enhancements can produce a successful agent, but the ablation shows that their interaction is not strictly additive. Therefore, the final discussion should distinguish between final capability and sample efficiency: the combined model is capable of solving Pong, but Double DQN only and multi-step return only show better sample efficiency in this particular ablation.

4. Additional analysis

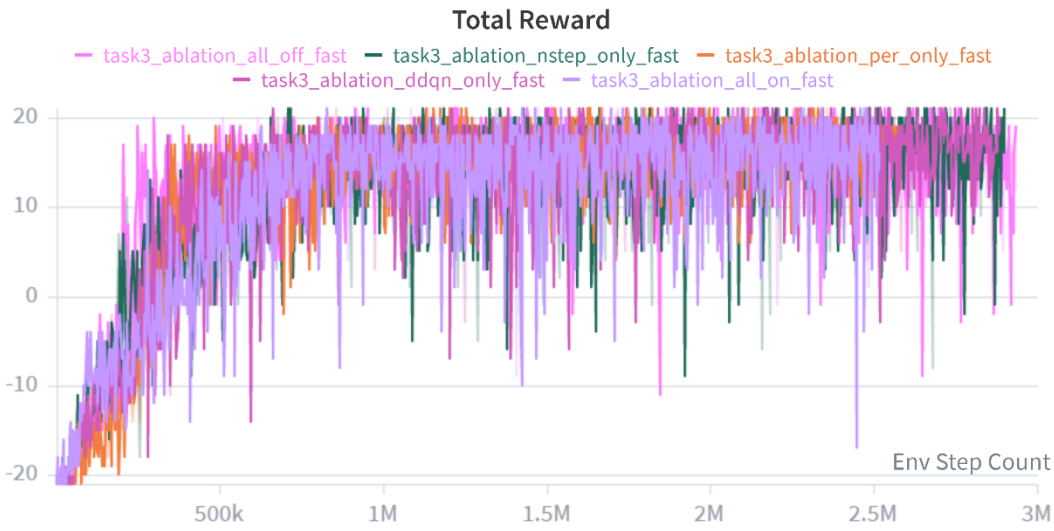

Figure 18. Training reward curves under the faster-exploration setting.

In addition to the main ablation study, I also conducted a faster-exploration experiment to investigate whether the agent could reach an average evaluation reward of 19 earlier, especially around the 600k-step checkpoint. In this setting, the exploration period was shortened so that the agent relied on exploitation much earlier. As shown in Figure 18, this strategy produced a visually promising early-stage training curve. Compared with the original ablation curves, the training rewards increased much faster, and several runs reached positive rewards within the first few hundred thousand environment steps. This confirms that reducing exploration can make the learning curve look better in the short term.

However, the later behavior shows a clear robustness problem. Although the fast setting improves early reward growth, the curves do not continue to improve reliably after the initial jump. The training rewards remain noisy, and occasional sharp drops still appear even after the agent has entered the positive-reward range. This suggests that the agent quickly learns a partially effective Pong policy, but this policy is not robust enough to

keep improving toward a near-optimal solution. In other words, the faster-exploration setting improves early exploitation, but it weakens the diversity and stability of the experience collected during the early training phase.

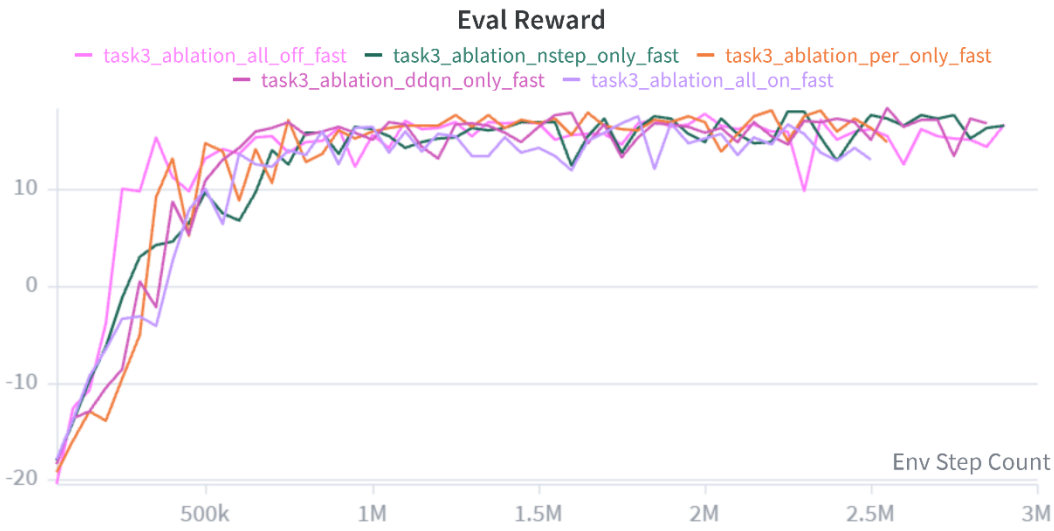

Figure 19. Evaluation reward curves under the faster-exploration setting.

Figure 19 provides a clearer view of this issue. The evaluation rewards improve much earlier than in the original setting, but they plateau below the target score. None of the fast-exploration variants reaches an average evaluation reward of 19 within the plotted range. The best values remain around 17.5 to 18.35, which is close to the target but still insufficient. This means that the early learning speed is not the same as final robustness. A policy can improve quickly in the beginning but still fail to become strong enough across the fixed 20-seed evaluation protocol.

Setting	Eval @ 600k	Eval @ 1M	Best Eval ≤ 3M	Step of Best Eval	First Eval ≥ 15	First Eval ≥ 19
All off fast	13.65	15.60	17.80	2.00M	350k	Not reached
Double DQN only fast	14.20	15.10	**18.35**	2.55M	650k	Not reached
PER only fast	8.75	15.90	18.05	2.20M	750k	Not reached
Multi-step only fast	6.75	16.20	17.95	2.25M	800k	Not reached
All on fast	13.65	**16.40**	17.55	1.80M	850k	Not reached

Table 5. Evaluation summary under the faster-exploration setting.

The statistics in Table 5 show the main trade-off. The all-off fast setting already reaches an evaluation reward of 13.65 at 600k, and the Double DQN only fast setting reaches 14.20 at the same step. These results are much better than the weak 600k checkpoints observed in the original long-exploration Task 3 run. However, the improvement saturates early. Even after training beyond 2M steps, the best fast-setting results remain below 19. This indicates that the shortened exploration schedule improves short-term sample efficiency but hurts long-term robustness.

This behavior also matches the Q-value diagnostics observed during training. In the faster setting, the Q-value estimates showed larger variance, meaning that the model's action-value predictions were less stable. For a game like Pong, this instability matters because the action values for moving up, moving down, or staying still can be very close in many states. If the Q-values fluctuate too much, the greedy action can change abruptly, which makes the paddle behavior less reliable. Therefore, the failure mode is

not simply that the agent does not learn; rather, it learns a usable but fragile policy that cannot consistently improve to the 19-point level.

This result suggests that the exploration phase is important for robustness. In early Atari training, the replay buffer is still being filled with diverse experiences. If exploration is reduced too aggressively, the buffer may contain a narrower distribution of states and actions. The agent may then overfit to a limited set of trajectories that produce fast early improvement, but it does not collect enough corrective experiences to handle harder or less frequent game situations. This explains why the fast curves look strong early but become less convincing later.

This observation also helps explain why PER alone was not very effective in the previous ablation study. PER samples transitions more frequently when they have larger temporal-difference errors, so it can improve learning efficiency when high-error transitions correspond to useful learning signals. However, during the early exploration phase, large TD errors may also come from noisy, unstable, or unrepresentative behavior. If PER emphasizes those transitions too early, the replay distribution may focus on errors produced by a weak exploratory policy rather than on transitions that genuinely improve the final policy. This is a plausible reason why PER alone improved more slowly than Double DQN or multi-step return in the main ablation study. Prioritized Experience Replay is designed to replay important transitions more often, using TD error as the priority signal, but this also means that the quality of the early replay distribution matters.

Overall, the faster-exploration experiment shows that early performance is not a sufficient indicator of robustness. A good Pong agent must not only increase reward quickly, but also maintain stable value estimates and continue improving under fixed evaluation seeds. The fast setting was useful as a diagnostic experiment because it revealed the cost of reducing exploration too much: faster early learning, but weaker long-term generalization and less stable Q-value behavior. Therefore, for the final Task 3 setting, I kept a longer exploration phase and evaluated robustness using fixed seeds 0–19 rather than selecting favorable seeds. This makes the reported performance more conservative and more comparable across Tasks 1–3.